# API: pruebas de servicios web con Postman

Registro e inicio de sesión

GA7-220501096-AA5-EV02

Laura Tatiana Ciro Botero

Servicio Nacional de Aprendizaje - SENA

Tecnología en Análisis y Desarrollo de Software

Ficha 3235898

Instructora Adriana Milena Rangel

2026

## Introducción, objetivo y alcance

Una API permite que una aplicación utilice funciones de otro programa mediante solicitudes. En esta evidencia se comprueban los servicios de registro e inicio de sesión desarrollados en AA5-EV01. Postman actúa como cliente y Spring Boot procesa las peticiones en el computador local.

La guía solicita instalar Postman, realizar el testing mediante un video, documentar las pruebas con pantallazos y entregar los endpoints. El archivo comprimido debe contener proyecto, video, documento de pruebas y archivos con endpoints y enlace del repositorio (Servicio Nacional de Aprendizaje [SENA], s. f., p. 15).

### Objetivo general

Verificar el comportamiento de los servicios web mediante casos correctos e incorrectos, conservar los resultados reales en Postman y entregar los recursos necesarios para repetir las pruebas.

### Alcance de la comprobación

Se ejecutan diez solicitudes con treinta comprobaciones en Postman de escritorio 12.35.4. Se valida el código HTTP, el mensaje esperado y la ausencia de los campos contrasena y password_hash en la respuesta. Se incluyen pruebas del registro, la autenticación y sus validaciones.

El caso confirma credenciales y devuelve un mensaje; no genera tokens ni crea una sesión con permisos dentro del sistema de selección de personal. La cuenta utilizada es ficticia. Los candidatos y datos personales del proyecto no se incluyen en estas pruebas.

## Endpoints y documentación del registro

Dirección base local: **http://127.0.0.1:8080**. Es necesario iniciar el servidor antes de enviar peticiones. GitHub conserva el código; no aloja esta API como servicio público.

**Tabla 1**
*Endpoints implementados*

| Método y ruta | Función | Respuestas |
|---|---|---|
| GET /api/salud | Comprobar disponibilidad. | 200 |
| POST /api/registro | Crear una cuenta única. | 201, 400, 409 |
| POST /api/login | Verificar las credenciales. | 200, 400, 401 |

### Registro: POST /api/registro

Encabezado: Content-Type: application/json. No exige un token previo. Recibe exactamente usuario y contrasena, ambos de tipo texto.

```
{
  "usuario": "tatiana.demo",
  "contrasena": "Practica2026!"
}
```

Respuesta correcta: HTTP 201 Created

```
{
  "estado": "registro_exitoso",
  "mensaje": "Usuario registrado correctamente.",
  "usuario": "tatiana.demo"
}
```

Si el usuario existe: HTTP 409 con el mensaje «El nombre de usuario ya está registrado.». Los datos inválidos devuelven 400. El almacenamiento usa BCrypt y una restricción UNIQUE para el nombre normalizado.

### Disponibilidad: GET /api/salud

```
{"estado":"disponible"}
```

## Inicio de sesión y reglas de validación

### Inicio de sesión: POST /api/login

Recibe el mismo JSON del registro, con Content-Type: application/json. Utiliza las credenciales de una cuenta registrada previamente. La autenticación correcta devuelve HTTP 200:

```
{
  "estado": "autenticacion_satisfactoria",
  "mensaje": "Autenticación satisfactoria.",
  "usuario": "tatiana.demo"
}
```

Un usuario inexistente o una contraseña incorrecta producen HTTP 401:

```
{"estado":"error", "mensaje":"Error en la autenticación."}
```

**Tabla 2**
*Reglas del contrato*

| Dato o condición | Regla |
|---|---|
| Usuario | De 3 a 30 letras ASCII, números, puntos, guiones o guiones bajos. Se quitan espacios exteriores y se convierte a minúsculas. |
| Contraseña | De 8 a 64 caracteres y hasta 72 bytes UTF-8. Conserva espacios y mayúsculas. |
| Campos obligatorios | Usuario y contraseña no pueden estar vacíos. |
| Campos adicionales o JSON incorrecto | HTTP 400. No se aceptan campos de rol o permisos. |
| Cuenta repetida | HTTP 409; no se crea otra cuenta con el mismo nombre. |
| Respuesta | No incluye la contraseña ni el hash almacenado. |

Los errores de validación contienen estado y mensaje; los de Bean Validation incluyen además una lista errores. No se depende del orden de esa lista para las pruebas.

# Procedimiento para repetir las pruebas

## 1. Descomprimir la entrega

Abrir Laura_Tatiana_Ciro_Botero_AA5_EV02.zip. Mantener juntos el archivo INICIAR_API.command y la carpeta ejecutable.

## 2. Iniciar la API

En este Mac, abrir INICIAR_API.command con doble clic. Dejar abierta su ventana. El lanzador usa Java 17 preparado en la carpeta de trabajo. En otro equipo se requiere Java 17 o superior.

## 3. Comprobar la disponibilidad

Enviar GET a http://127.0.0.1:8080/api/salud. El resultado debe ser 200 y estado disponible. Si ya existe un servidor funcionando en 8080, no iniciar una segunda copia.

## 4. Importar la colección

En Postman, usar Import y elegir proyecto/postman/AA5_EV02.postman_collection.json. El archivo contiene las diez solicitudes, el cuerpo JSON y los scripts de verificación.

## 5. Revisar las variables

baseUrl tiene el valor http://127.0.0.1:8080. usuarioPrueba y semilla se generan al enviar la solicitud 02; no es necesario escribirlas manualmente.

## 6. Enviar las peticiones

Ejecutar de 01 a 10 en orden. En Body se ve el JSON enviado; en la zona inferior se ve el código y la respuesta. La contraseña de prueba es ficticia.

## 7. Ejecutar el Runner

Abrir el menú de la colección, elegir Run, mantener Functional, Local, una iteración y las diez peticiones seleccionadas. Pulsar Start run.

## 8. Leer los resultados

Comprobar Passed 30, Failed 0 y Errors 0. Un 400, 401 o 409 previsto es una prueba aprobada; se comprueba que el servidor rechace correctamente esa petición.

Para compilar desde el código: entrar en proyecto, ejecutar mvn clean verify y usar el JAR generado. Para detener el servidor: Ctrl+C en su ventana. No borrar data si se desean conservar las cuentas de práctica.

## Plan de pruebas y resultados observados

**Tabla 3**
*Matriz ejecutada en Postman*

| Caso y entrada | HTTP esperado / obtenido | Resultado |
|---|---|---|
| 01. GET de disponibilidad, sin cuerpo. | 200 / 200 | 3 de 3 |
| 02. Usuario nuevo y clave Practica2026!. | 201 / 201 | 3 de 3 |
| 03. Credenciales del usuario recién creado. | 200 / 200 | 3 de 3 |
| 04. Clave incorrecta Incorrecta2026!. | 401 / 401 | 3 de 3 |
| 05. Repetir el usuario del caso 02. | 409 / 409 | 3 de 3 |
| 06. Usuario no.existe seguido de la semilla. | 401 / 401 | 3 de 3 |
| 07. Usuario y contraseña vacíos. | 400 / 400 | 3 de 3 |
| 08. Contraseña 123, menor de 8 caracteres. | 400 / 400 | 3 de 3 |
| 09. Incluir el campo no permitido rol: ADMIN. | 400 / 400 | 3 de 3 |
| 10. Contraseña de 40 letras á: 80 bytes UTF-8. | 400 / 400 | 3 de 3 |

Resultado observado: **30 comprobaciones aprobadas, 0 fallidas, 0 omitidas y 0 errores**. El Runner completó una iteración en 3 s 117 ms y mostró 193 ms como tiempo medio de respuesta. Son mediciones de esta ejecución local, no una prueba de rendimiento o capacidad.

Antes del Runner también se enviaron individualmente las diez solicitudes. Las capturas originales están en evidencias/postman. Los nombres de usuario varían entre la prueba individual y la ejecución completa, porque cada nuevo registro genera otra semilla.

## Scripts de verificación y repetibilidad

Los scripts de Post-response se ejecutan después de recibir una respuesta. pm.test define una comprobación y pm.expect compara el dato recibido con el esperado. Los resultados se consultan en Test Results (Postman, s. f.-b).

### Ejemplo del caso de inicio de sesión correcto

```
pm.test("HTTP 200", () =>
  pm.response.to.have.status(200));
const datos = pm.response.json();
pm.test("Respuesta esperada", () =>
  pm.expect(datos.mensaje)
    .to.eql("Autenticación satisfactoria."));
pm.test("No expone claves ni hash", () => {
  pm.expect(datos).not.to.have.property("contrasena");
  pm.expect(datos).not.to.have.property("password_hash");
});
```

### Preparación del registro para repetir la colección

```
// Cada ejecución genera una cuenta de prueba nueva.
const semilla = Date.now().toString();
pm.collectionVariables.set("semilla", semilla);
pm.collectionVariables.set("usuarioPrueba", "taty." + semilla);
```

La solicitud 02 crea el usuario. La 03 lo utiliza para autenticarse; la 05 intenta duplicarlo. Por eso se mantiene el orden. La comprobación de ausencia de claves revisa estos dos campos en el objeto de respuesta; no sustituye una auditoría completa de seguridad.

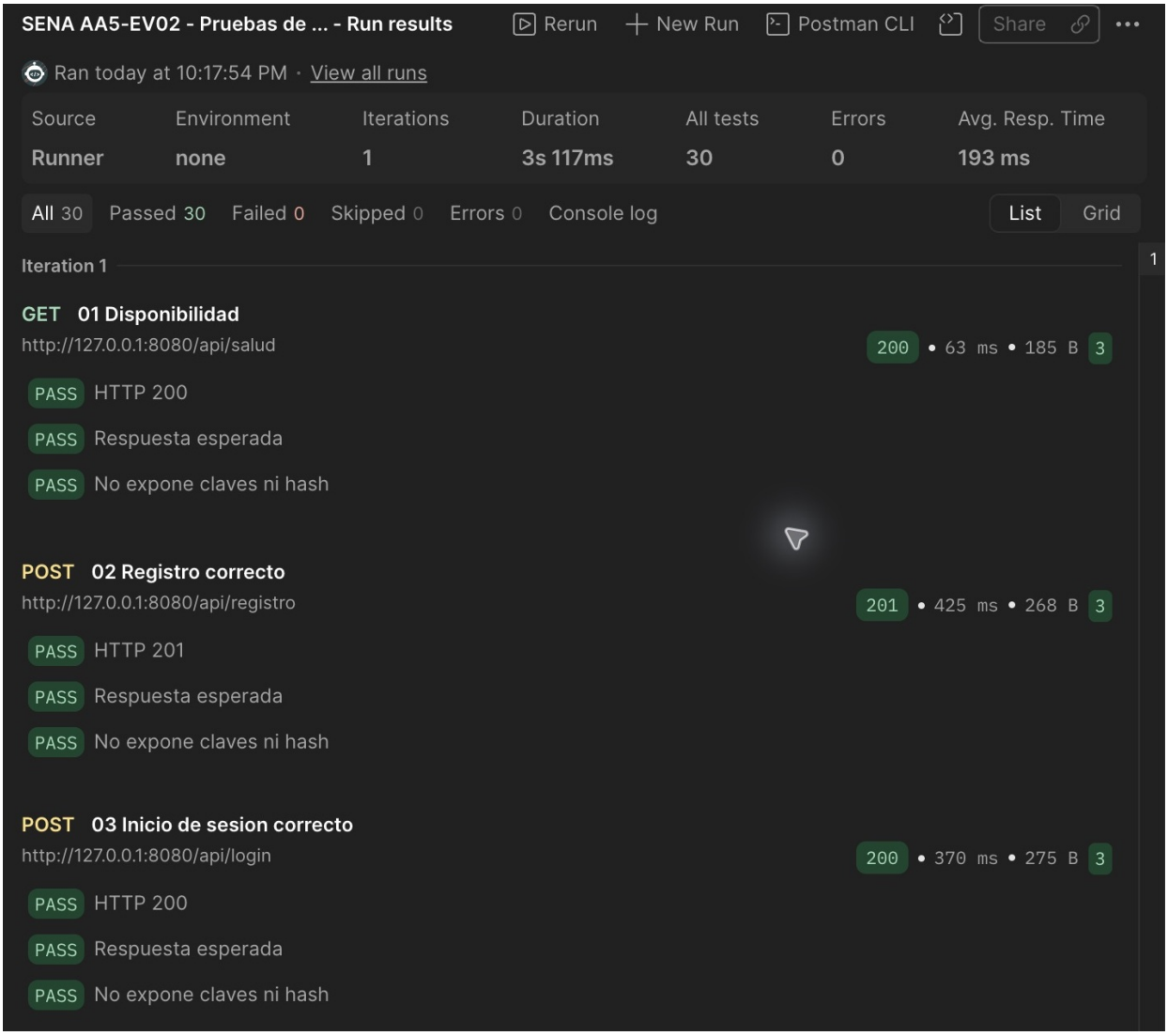
### Verificación complementaria

La misma colección también se ejecutó con Newman en GitHub Actions: 10 solicitudes y 30 comprobaciones aprobadas. Maven aprobó 15 pruebas del servidor. Newman permite automatizar colecciones de Postman, pero aquí se conserva además la demostración real de la aplicación de escritorio (Postman, s. f.-a).

## Resultado real del Runner de Postman

**Figura 1**

*Treinta comprobaciones aprobadas en la aplicación de escritorio*



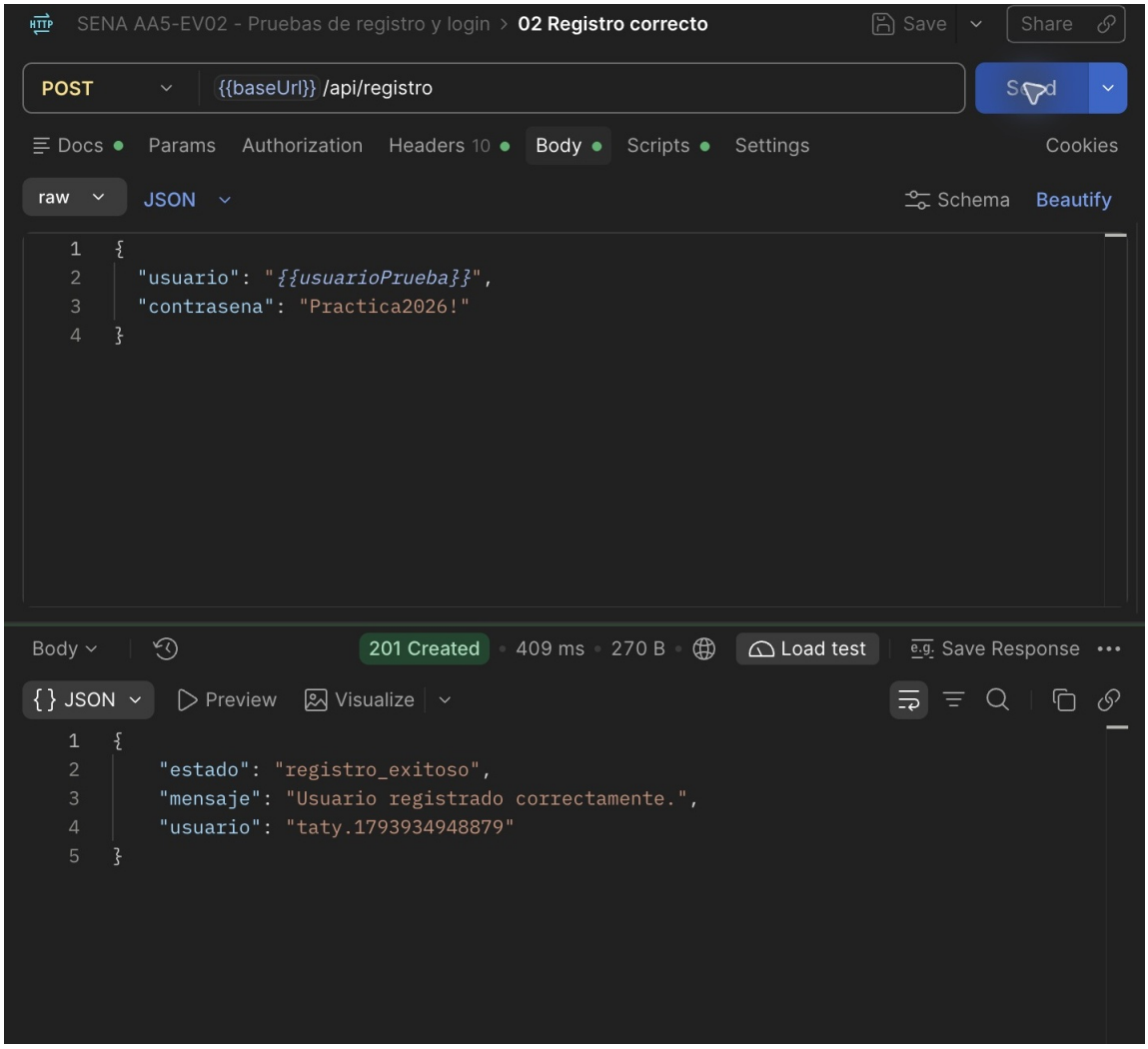

Nota. Captura real recortada al panel central para facilitar la lectura. La cabecera muestra Passed 30, Failed 0, Skipped 0 y Errors 0. Se observan las tres verificaciones aprobadas de cada solicitud. El archivo original sin recorte se incluye en evidencias/postman.

El rechazo esperado de una petición inválida también aparece como PASS. Por ejemplo, la solicitud de contraseña incorrecta pasa si recibe HTTP 401 y el mensaje definido por el contrato.

## Evidencia del registro correcto

**Figura 2**

*Petición JSON y respuesta HTTP 201 del registro*



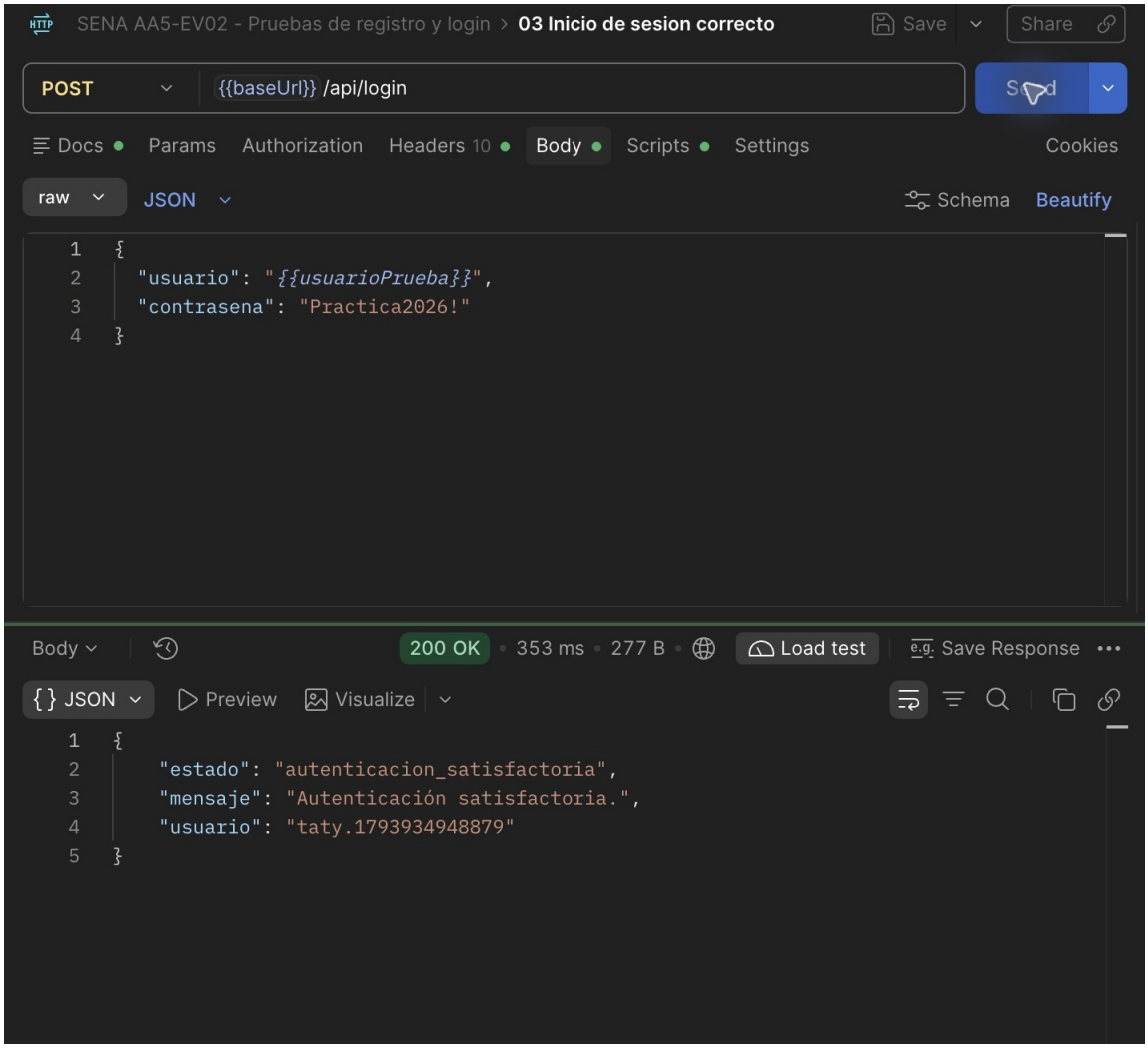

Nota. Captura real del caso 02. El cuerpo incluye las variables de la colección y una contraseña ficticia. La respuesta confirma el registro y devuelve el nombre del usuario, sin contraseña ni hash.

Interpretación: el servidor recibió información válida, creó la cuenta y respondió con 201 Created. La variable usuarioPrueba conserva el nombre para las solicitudes siguientes.

## Evidencia del inicio de sesión correcto

**Figura 3**

*Autenticación satisfactoria y respuesta HTTP 200*



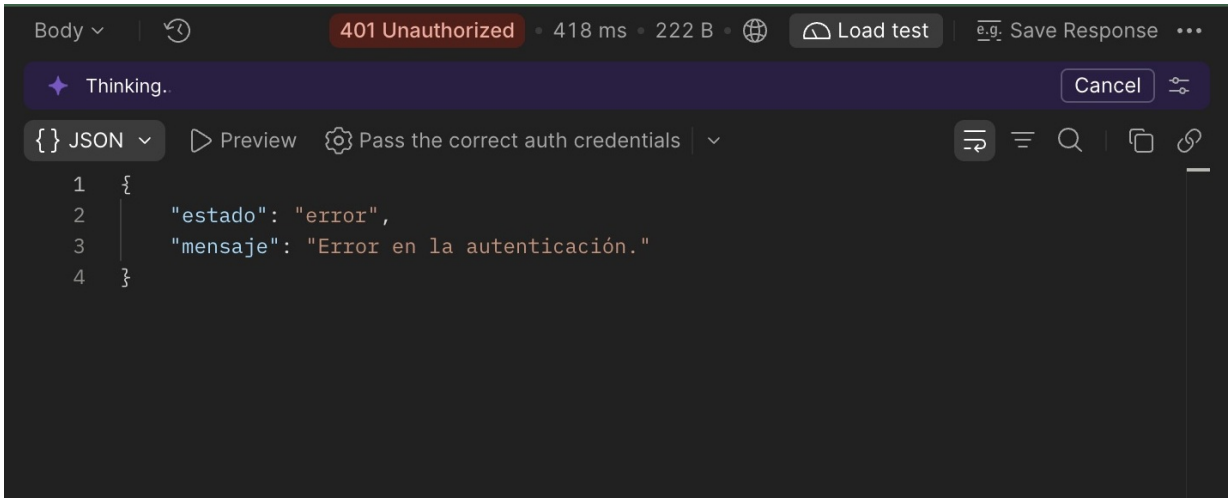

Nota. Captura real del caso 03. La solicitud reutiliza las credenciales del registro. El resultado muestra autenticacion_satisfactoria y el mensaje solicitado.

Interpretación: la contraseña enviada coincide con el hash de la cuenta. El resultado cumple el caso de autenticación; no debe interpretarse como la creación de una sesión persistente o un permiso de acceso.

## Errores previstos: credenciales y duplicados

**Figura 4**

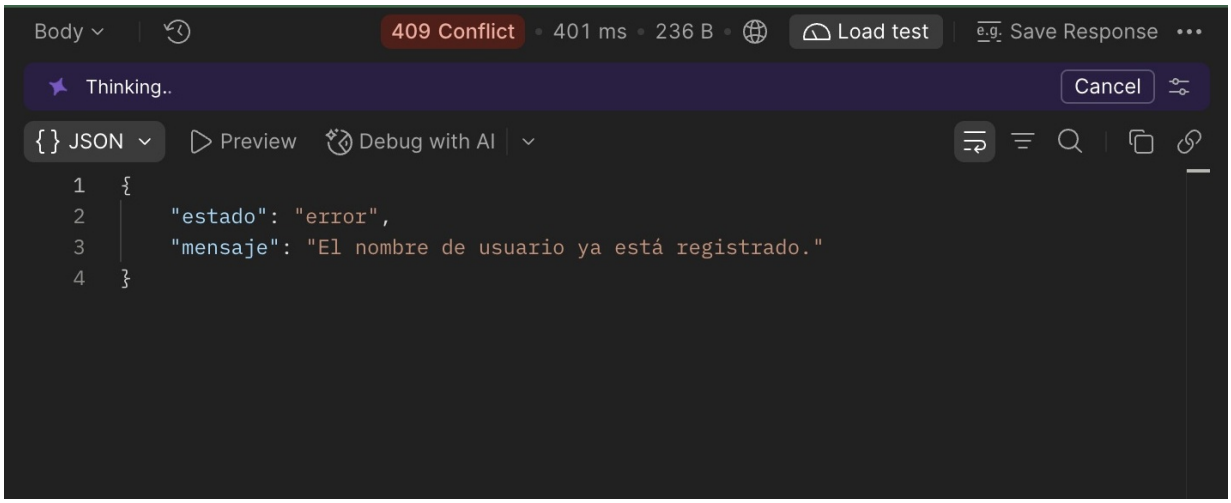
*Contraseña incorrecta: HTTP 401*




Nota. Caso 04: se envió Incorrecta2026! al servicio /api/login. El mensaje es «Error en la autenticación.». El caso 06 de usuario inexistente devuelve el mismo código y mensaje; su captura completa también se entrega.

**Figura 5**

*Registro duplicado: HTTP 409*




Nota. Caso 05: se repitió el nombre creado por el caso 02. El servidor indicó que ya estaba registrado. Los dos resultados corresponden a rechazos correctos y sus comprobaciones pasaron.

## Validación: obligatoriedad y longitud

**Figura 6**

*Campos obligatorios vacíos: HTTP 400*




Nota. Caso 07: usuario y contrasena se enviaron vacíos. La respuesta enumera las validaciones incumplidas. No se realizó un registro válido.

**Figura 7**

*Contraseña de tres caracteres: HTTP 400*

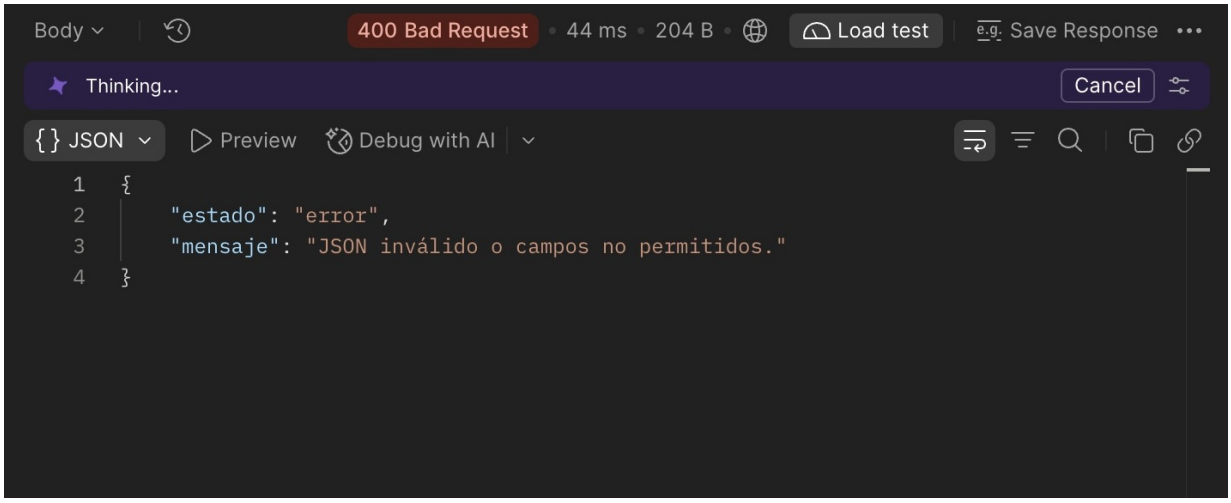



Nota. Caso 08: el valor 123 incumple la longitud mínima. La respuesta contiene «Datos inválidos.» y la indicación de longitud. Se comprobó el rechazo antes de persistir la cuenta.

## Validación: contrato JSON y límite UTF-8

**Figura 8**

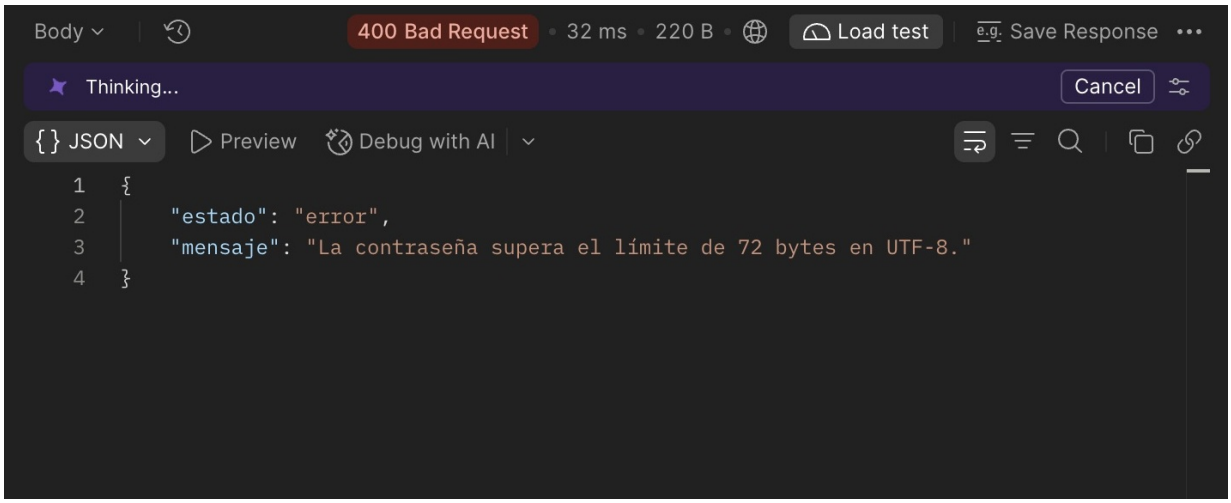
*Campo rol no permitido: HTTP 400*




Nota. Caso 09: se agregó rol con valor ADMIN al cuerpo. La API rechazó el campo adicional con el mensaje «JSON inválido o campos no permitidos.».

**Figura 9**

*Contraseña mayor de 72 bytes: HTTP 400*




Nota. Caso 10: cuarenta letras á ocupan ochenta bytes UTF-8, aunque solo sean cuarenta caracteres. El servidor no las trunca: rechaza la contraseña por superar el límite de 72 bytes.

## Video, repositorio y contenido de la entrega

Repositorio: sena-ga7-aa5-ev02-api

Dirección completa: https://github.com/tatianabotero314-del/sena-ga7-aa5-ev02-api

Ejecución automatizada: Abrir el reporte de Maven y Newman

Versión del código comprobado: c2112cbf2716f1c6afb65c4a96c03369ec88d673. El historial de Git está incluido en historial-github.bundle. El proyecto contiene comentarios y el flujo de verificación de GitHub Actions.

### Video incluido

Abrir VIDEO_Pruebas_Postman_AA5_EV02.mp4. Muestra la colección, las diez solicitudes, sus resultados y el Runner. Se elaboró a partir de capturas secuenciales reales de la ventana de Postman, con recorte del panel lateral, pausas de lectura y explicaciones en español mediante subtítulos. El video no contiene narración oral.

| Archivo o carpeta | Contenido |
|---|---|
| PDF de esta evidencia | Portada, documentación, procedimiento, capturas, resultados y lista de chequeo. |
| proyecto/ | Código Java, configuración, SQL, pantalla de práctica, pruebas y colección Postman. |
| VIDEO_Pruebas_Postman_AA5_EV02.mp4 | Demostración subtitulada de las pruebas reales. |
| ENDPOINT.txt / ENLACE_GITHUB.txt | Direcciones de la API y del repositorio. |
| ejecutable/ e INICIAR_API.command | JAR compilado y lanzador para este Mac. Requiere Java 17 o superior. |
| evidencias/ | Capturas originales, resultados de Postman y reportes complementarios de Maven y Newman. |
| GUION_VIDEO.txt y LEEME.txt | Explicación del video e instrucciones de uso. |

ZIP: Laura_Tatiana_Ciro_Botero_AA5_EV02.zip. No incluye la base de cuentas de práctica ni las herramientas descargadas. El código y la colección permiten reconstruir y repetir el caso.

## Lista de chequeo y conclusiones

**Tabla 4**
*Autoevaluación de los requisitos de AA5-EV02*

| Indicador | Sí | No | Evidencia de cumplimiento |
|---|---|---|---|
| 1. Realiza el test de la API utilizando Postman. | X | | Diez solicitudes ejecutadas en Postman de escritorio; Runner con 30 aprobadas, 0 fallidas y 0 errores. Tabla 3 y figura 1. |
| 2. Realiza el video solicitado mostrando las pruebas del API. | X | | MP4 con captura secuencial de las solicitudes, respuestas y resumen del Runner, con explicación en español. |
| 3. Realiza la documentación del API realizada. | X | | Contrato, entradas, respuestas, validaciones, procedimiento, matriz y pantallazos en este PDF. |
| 4. Entrega los endpoints de las API desarrolladas. | X | | Tabla 1 y archivo ENDPOINT.txt con las tres direcciones completas. |
| Requisitos de empaquetado de la guía. | X | | ZIP con proyecto, video, PDF, endpoints y enlace del repositorio. Código versionado en GitHub. |

Nota. Lista de autoevaluación del producto; la calificación corresponde a la instructora. El cumplimiento se sustenta en los archivos y resultados entregados.

### Conclusiones

Los servicios respondieron según el contrato tanto en los casos correctos como en los errores previstos. El registro creó una cuenta única; el inicio de sesión distinguió credenciales válidas e inválidas; y las validaciones rechazaron entradas incompletas, campos ajenos al contrato y exceso de bytes.

La colección y sus variables permiten repetir el ejercicio sin reescribir peticiones. El video, los pantallazos y los reportes conservan evidencia de la ejecución. Las pruebas verifican este caso académico; no representan una evaluación integral de un sistema en producción.

## Referencias

Postman. (s. f.-a). *Newman CLI overview*. https://learning.postman.com/docs/reference/newman-cli/command-line-integration-with-newman/

Postman. (s. f.-b). *Write scripts to test API response data in Postman*. https://learning.postman.com/docs/tests-and-scripts/write-scripts/test-scripts/

Servicio Nacional de Aprendizaje. (s. f.). *Guía de aprendizaje 7: Actividad GA7-220501096-AA5* (p. 15) [Material de formación proporcionado para la actividad].

Servicio Nacional de Aprendizaje. (2026). *Estado: Arranca GA7-AA5* [Diapositiva de clase, captura proporcionada por la aprendiz].

**Presentación del documento**

Documento académico basado en APA: tamaño carta, márgenes de 2,54 cm, familia Times de 12 puntos, interlineado doble en el cuerpo, numeración de páginas, títulos de tablas y figuras y referencias con sangría francesa. Se ajustó el interlineado de tablas, notas y código para facilitar su lectura.

Las figuras provienen de la ejecución real en Postman. Los recortes conservan las respuestas y no modifican los resultados. Las capturas originales completas permanecen en el paquete.